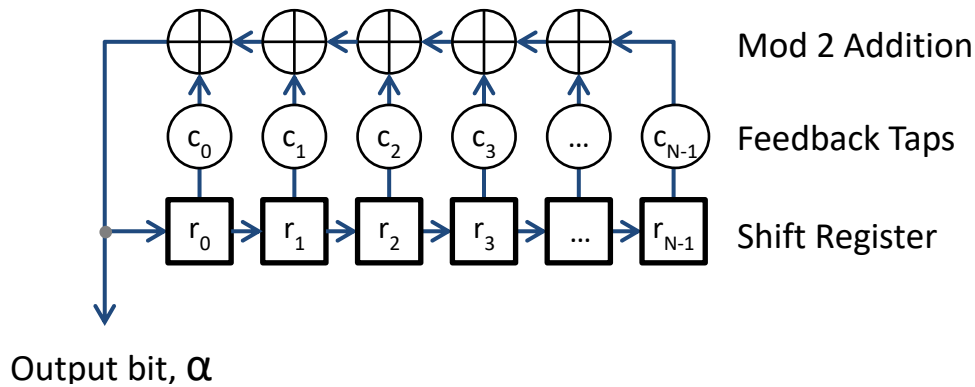


Primer: Linear Feedback Shift Registers

We would like to create a mathematical model of an LFSR in order to predict and analyse its behaviour. The first step is to define a notation. Any notation will work so long as we are extremely careful to apply it consistently; however the following one is convenient.

Consider an N -bit binary shift register with an external linear feedback connection:



The output bit α is the linear sum of the register bits selected by the constant tap bits $c_0 \dots c_{N-1}$. At each clock interval the register contents move one bit to the right and r_0 takes on the value of the previous output.

We can model the feedback network in modulo-2 arithmetic as follows:

$$\begin{aligned}\alpha &= c_0 r_0 + c_1 r_1 + \dots + c_{N-1} r_{N-1} \\ &= \sum_{k=0}^{N-1} c_k r_k\end{aligned}\tag{1}$$

It is evident that the register bits hold the N previous values of the output, so if we denote the value of register bit b at time t as $r_{b,t}$ and the output bit at time t as α_t then we can say:

$$\begin{aligned}r_{0,t} &= \alpha_{t-1} \\ r_{1,t} &= \alpha_{t-2} \\ &\dots \\ r_{N-1,t} &= \alpha_{t-N}\end{aligned}\tag{2}$$

From (1) and (2) we can write an expression for the current output bit in terms of the previous values:

$$\alpha_t = c_0 \alpha_{t-1} + c_1 \alpha_{t-2} + \dots + c_{N-1} \alpha_{t-N}\tag{3}$$

Typically it is more useful to re-state (3) to obtain an expression for the *next* output bit:

$$\alpha_{t+1} = c_0 \alpha_t + c_1 \alpha_{t-1} + \dots + c_{N-1} \alpha_{t-(N-1)}\tag{4}$$

This is a useful result. However we would prefer a single expression that models both the motion of the shift register and the calculation of the next output bit. We can achieve this if we use vector and matrix notation. Note that this is merely a compact way of representing a system of simultaneous linear equations.

We start by representing the state of the register at time t as an N bit column vector:

$$\overline{R}_t = \begin{bmatrix} r_{0,t} \\ r_{1,t} \\ \vdots \\ r_{N-1,t} \end{bmatrix} = \begin{bmatrix} \alpha_{t-1} \\ \alpha_{t-2} \\ \vdots \\ \alpha_{t-N} \end{bmatrix}$$

If we now construct the $N \times N$ square matrix A as follows:

$$A = \begin{pmatrix} c_0 & c_1 & \dots & c_{N-1} \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \vdots \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

By the normal rules of matrix multiplication we can readily see that applying the matrix A moves every element of $\overline{R}$ down one and replaces the first element with the required sum of products.

This provides an important result that can be applied in several very useful ways:

$$\overline{R}_{t+1} = A \cdot \overline{R}_t \quad (5)$$

A common objective is to derive the tap bits from a known sample of the output sequence. We can approach this as follows.

Consider the following expression:

$$\begin{pmatrix} \alpha_{t-1} & \alpha_{t-2} & \dots & \alpha_{t-N} \\ \alpha_{t-2} & \alpha_{t-3} & \dots & \alpha_{t-N-1} \\ \vdots & & & \vdots \\ \alpha_{t-N} & \alpha_{t-N-1} & \dots & \alpha_{t-2N+1} \end{pmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} \alpha_t \\ \alpha_{t-1} \\ \vdots \\ \alpha_{t-(N-1)} \end{bmatrix} \quad (6)$$

This forms an $N \times N$ matrix from the previous $2N - 1$ values of the output sequence¹ and applies it to a column vector of the tap bits. It is readily seen that the result is a column vector of the *current* output bit and the $N - 1$ previous values.

If we multiply by the *inverse*² of the matrix then we obtain an expression that enables us to calculate the tap bits from any consecutive $2N$ samples of the output sequence:

$$\begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{pmatrix} \alpha_{t-1} & \alpha_{t-2} & \dots & \alpha_{t-N} \\ \alpha_{t-2} & \alpha_{t-3} & \dots & \alpha_{t-N-1} \\ \vdots & & & \vdots \\ \alpha_{t-N} & \alpha_{t-N-1} & \dots & \alpha_{t-2N+1} \end{pmatrix}^{-1} \cdot \begin{bmatrix} \alpha_t \\ \alpha_{t-1} \\ \vdots \\ \alpha_{t-(N-1)} \end{bmatrix} \quad (7)$$

¹ in linear algebra this type of construction is known as a [Hankel matrix](#)

² To be precise the modular inverse, calculated via (for example) a mod-2 version of the [Gauss-Jordan method](#)

It may be helpful to show a worked example.

Suppose we have twelve consecutive samples of a sequence generated by a 6-bit LFSR³:

α_{t-11}	α_{t-10}	α_{t-9}	α_{t-8}	α_{t-7}	α_{t-6}	α_{t-5}	α_{t-4}	α_{t-3}	α_{t-2}	α_{t-1}	α_t
1	0	0	0	1	0	1	0	0	1	1	1

Equation (6) becomes:

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

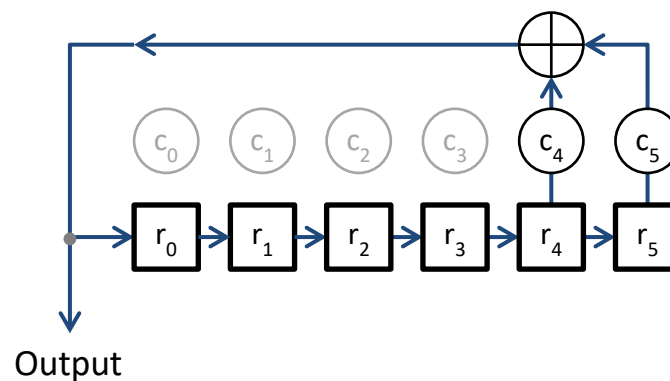
Multiplying by the modular inverse matrix calculated using my [JavaScript tool](#) gives:

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 \end{pmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

So in our chosen notation the sequence can be generated with taps:

$$c_0 = 0, c_1 = 0, c_2 = 0, c_3 = 0, c_4 = 1, c_5 = 1$$



³ note that as we (conventionally) read from left to right, the most recent value is the one on the **right**

We can validate that this is correct. If we start from the all-1's register state then the LFSR produces the following repetitive 63 bit pattern, which contains the required sequence (highlighted):

000001000011000101001111010001110010010110111011001101010111110

Note that our solution doesn't disclose the point in the sequence at which the desired sample occurs. However as we know that the register always contains the *preceding* six output values, we can say that when the LFSR is outputting the final bit of the sample the register bits are $r_0 = 1$, $r_1 = 1$, $r_2 = 0$, $r_3 = 0$, $r_4 = 1$ and $r_5 = 0$. If we now work backwards guessing each preceding output value in turn⁴, we can eventually determine that at the start of the sequence the register bits were $r_0 = 1$, $r_1 = 0$, $r_2 = 0$, $r_3 = 0$, $r_4 = 0$ and $r_5 = 1$. Starting from this initial state produces the desired result:

time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
output	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	0	0	0	1	1	1	...
r0	1	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	0	0	0	1	1	...
r1	0	1	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	0	0	0	1	...
r2	0	0	1	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	0	0	0	...
r3	0	0	0	1	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	0	0	...
r4	0	0	0	0	1	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	0	...
r5	1	0	0	0	0	1	1	0	0	0	1	0	1	0	0	1	1	1	1	0	1	...

Be very careful in your counting as it is surprisingly easy to be one bit out. Remember in particular that with our notation, the value of r_0 is the *previous* output bit not the current one.

v1.0

© copyleft MJCROSS 2018
all wrongs reserved

⁴ It is left as an exercise for the reader to find a more elegant way of stepping an LFSR backwards